

Lean Formalization of the Parameter Classification for Conditional Entropies

Roberto Rubboli, Erkka Haapasalo, and Marco Tomamichel

This note records the Lean formalization of the necessary-and-sufficient parameter classification for the concrete conditional-entropy candidates in the complete-proof paper, Ref. [2]. It gives a compact theorem-facing proof map, a statement-by-statement correspondence table, and the precise trust and scope boundary. The presentation is deliberately theorem-facing: it separates the public conclusions, their paper-level ingredients, and the proved reusable infrastructure.

1. LEAN CORRESPONDENCE

The formal development follows the complete analytic proofs in Ref. [2]. It covers the endpoint-aware Rényi parameter, real-power and entropy differentiation, signed integration, common discretization, the two-block, three-block, and Shannon localizers, exact tropical stationarity, all sufficiency and necessity branches, and the original finite semiring and conditionally mixing channel lift. The source is available at <https://github.com/robertorubboli/A-complete-characterization-of-conditional-entropies> on the repository’s main branch.

Throughout this note, every numbered lemma, proposition, or remark uses the number in the complete-proof paper, Ref. [2]. That paper alone determines the organization, numbering, and conventions of the Lean correspondence.

There are two terminal conclusions. The declaration `mainClassification` says that, for every probability measure on the compactified Rényi parameter space, monotonicity of each concrete candidate is equivalent to its exact admissibility condition. Its only explicit argument is the measure being classified; it has no auxiliary hypothesis. The closed declaration `allConditionalEntropyAxioms` proves nonnegativity, zero-embedding invariance, tensor additivity, fair-bit normalization, and embedded channel monotonicity for every admissible candidate. Neither theorem assumes a project-specific axiom. In the complete-proof paper and in Lean, `log` is the natural logarithm and an independent fair bit has value $\log 2$.

The repository contains two complementary dependency diagrams at different levels of detail. Figure 1 is the *theorem-facing proof map*: it connects public conclusions to numbered complete-proof results and reusable proof ingredients. It is not a module-import graph. The standalone files `paper/dependency-graph.dot`, `paper/dependency-graph.svg`, and `paper/dependency-graph.png` are three formats of one detailed *Lean implementation dependency graph*; their arrows run from prerequisite to dependent.

In Figure 1, arrows deliberately point in the reverse direction, from a conclusion to ingredients used in its proof. Pale green boxes are the two public conclusions, white boxes are paper-facing intermediate results, pale blue boxes are reusable infrastructure that is itself proved in Lean, and the dashed gray box is an upstream representation theorem cited from Section III of Ref. [1]. That gray result is needed only for the statement that every abstract derivation has the displayed barycentric form; it is not an axiom or hypothesis of either green Lean declaration.

Table I follows the numbering of the complete-proof paper, Ref. [2]. All Lean names are in the namespace `ConditionalEntropy`. A split API means that one complete-proof statement is intentionally proved by several smaller declarations. The last row is worded differently because Proposition 5.10 combines the formalized parameter restriction with the upstream representation theorem identified above.

The table records the human-facing correspondence. The repository also keeps an internal formalization-blueprint ledger with 152 source-ordered rows, together with the declaration index

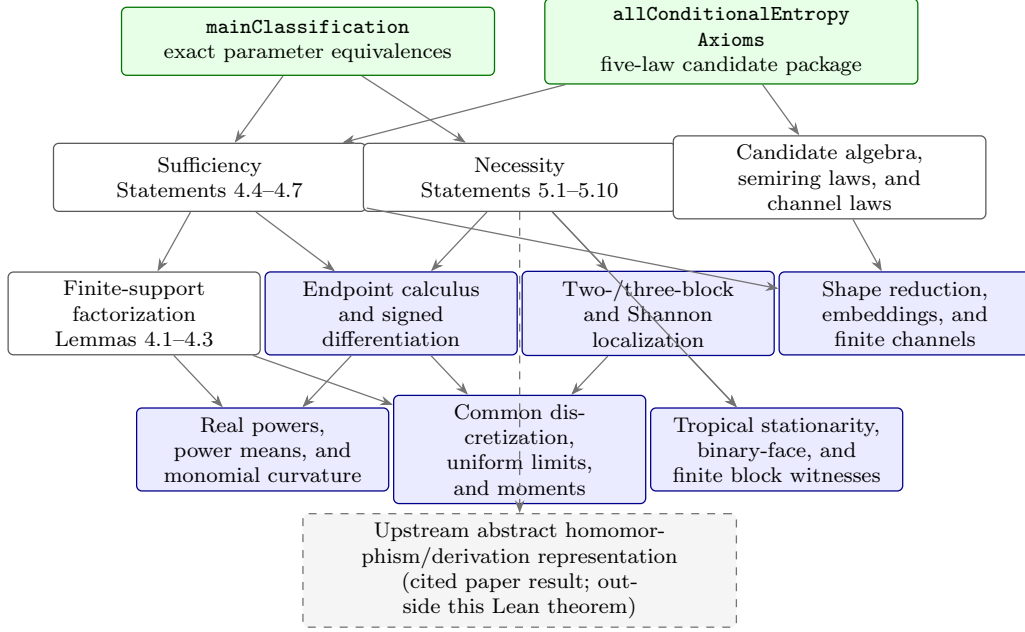


FIG. 1. Theorem-facing proof map for the parameter classification. Every displayed statement number is from the complete-proof paper, Ref. [2]. Arrows point from a result to ingredients used in its proof, the reverse of the prerequisite-to-dependent convention in the detailed Lean implementation dependency graph. Color records proof role, not confidence: green means public conclusion, blue means internally proved infrastructure, white means a paper-facing result, and dashed gray marks the explicitly excluded upstream representation theorem. The dashed connector records manuscript context, not a Lean axiom or dependency.

TABLE I. Statement-by-statement correspondence. All numbers are from the complete-proof paper, Ref. [2].

Complete-proof statement	Lean declaration	Status
Lemma 4.1: power-mean curvature	<code>finiteHolder; powerCurvature; parameterPowerCurvature</code>	Formalized
Lemma 4.2: monomial curvature	<code>affineMonomialCurvaturePackage</code>	Formalized
Lemma 4.3: finite-support sufficiency	<code>finiteSufficiency</code>	Formalized
Proposition 4.4: general temperate sufficiency	<code>generalSufficiency; finiteTSufficiency</code>	Formalized (split API)
Proposition 4.5: negative-tropical sufficiency	<code>negativeTropicalSufficiency</code>	Formalized
Proposition 4.6: positive-tropical sufficiency	<code>positiveTropicalComponent; positiveTropicalSufficiency</code>	Formalized (split API)

Table I (continued; complete-proof numbering).

Complete-proof statement	Lean declaration	Status
Proposition 4.7: derivation sufficiency	<code>derivationSufficiency</code>	Formalized
Proposition 5.1: positive necessity	<code>positiveNecessity;</code> <code>positiveTemperateProbabilityNecessity</code>	Formalized (split API)
Proposition 5.2: negative Shannon atom with upper support	<code>negativeShannonObstruction</code>	Formalized
Proposition 5.3: truncated exceptional-moment bound	<code>truncatedMoment;</code> <code>exceptionalUpperMeasurePackage;</code> <code>MLower_le_of_truncated_difference</code>	Formalized (split API)

Table I (continued; complete-proof numbering).

Complete-proof statement	Lean declaration	Status
Remark 5.4: homogeneity and channel lift	<code>columnPhi_posHomOne; columnDeriv_posHomOne;</code> <code>conditionalSemiringOfJointProb_le;</code> <code>embeddingLift</code>	Formalized (split API)
Remark 5.5: compensated two-column witness	<code>normalizedTwoColumn;</code> <code>normalizedTwoColumn_column_zero;</code> <code>normalizedTwoColumn_column_one;</code> <code>mergeTwoChannel_output_column;</code> <code>sum_normalizedTwoColumn</code>	Formalized (split API)
Proposition 5.6: complete exceptional negative branch (unique upper point, one-sided finiteness, and moment bound)	<code>oneUpperPoint;</code> <code>negativeTemperateNecessity_of_atom_zero</code>	Formalized (split API)

and editable Lean implementation dependency graph. Those longer audits are kept out of this note so that the main proof route remains readable.

A. Trust boundary and reproducibility

The project is pinned to Lean 4.32.2 and Mathlib 4.32.2. The release source contains no `sorry`, `admit`, project-defined `axiom`, or project-defined `opaque` declaration. The final declarations contain no unproved project hypothesis. Lean’s standard logical foundations, including classical choice, propositional extensionality, and quotient soundness, remain part of the ordinary trusted base.

The single command

```
powershell -File scripts/check.ps1
```

Table I (continued; complete-proof numbering).

Complete-proof statement	Lean declaration	Status
Proposition 5.7: negative-tropical necessity	<code>negativeTropicalUpperAtomZero;</code> <code>twoUpperSupportPointsTropicalObstruction;</code> <code>tropicalTruncatedMoment_of_exceptional;</code> <code>negativeTropicalNecessity</code>	Formalized (split API)
Proposition 5.8: positive-tropical necessity	<code>sameSupportDecomposition;</code> <code>probabilityEqDiracZero;</code> <code>positiveTropicalNecessity</code>	Formalized (split API)
Proposition 5.9: derivation necessity	<code>derivationNecessity</code>	Formalized
Proposition 5.10: represented derivations	<code>derivationSufficiency;</code> <code>derivationNecessity;</code> <code>derivation field of mainClassification</code>	Range proved; representation cited

runs the forbidden-token scan, a warning-as-error build of the integrated `CompleteCharacterization` target, the axiom audit, and a structural audit of the internal 152-row blueprint ledger. The latter checks row order, labels, and table shape; the human-facing 4.1–5.10 correspondence above was reviewed separately against the theorem statements. The formalization proves the classification and law package for the four concrete candidate families in the complete-proof paper’s natural-log convention. The upstream representation of every abstract homomorphism or derivation and the later barycentric classification of every abstract conditional entropy remain results of Ref. [1], not hidden assumptions of the Lean terminal theorems.

-
- [1] R. Rubboli, E. Haapasalo, and M. Tomamichel. “A complete characterisation of conditional entropies”, (2026). DOI: [10.48550/arXiv.2601.23213](https://doi.org/10.48550/arXiv.2601.23213).
- [2] R. Rubboli, E. Haapasalo, and M. Tomamichel. “Sufficient and Necessary Conditions for the Parameters of Conditional Entropies: Complete Proofs”. Auxiliary document on GitHub, (2026). Available online: <https://github.com/robertorubboli/A-complete-characterization-of-conditional-entropies/blob/main/paper/auxiliary-files/pdf/sections-4-5-full-details.pdf>. Complete proofs for Sections 4 and 5 of the canonical manuscript.